

Lab 3: Component Modelling & Architectural Pattern Selection

Name: P MANAS SIDDHARDHA
SRN: PES1UG24AM186

Architectural Justification

Multi-Cloud Asset Cost Optimization Portal — Problem Statement #45

Architecture Selection: We chose Microservices Architecture for the Multi-Cloud Asset Cost Portal
Architectural Choice

The portal is decomposed into independently deployable microservices Billing Ingestion, Cost Classification, Idle Asset Detection, Alerting & Notification, a Dashboard/Reporting API, and a shared Billing Data Repository communicating over REST/event-driven interfaces rather than as a single monolithic application or a simple client-server system.

Two Scenario-Related Reasons

- 1. Independent scaling for heterogeneous, high-volume ingestion :** AWS, Azure, and GCP billing exports arrive in different formats, volumes, and schedules, and NFR-001 requires multi-gigabyte files to be parsed and dashboards refreshed in under two minutes. Microservices let the Billing Ingestion Service scale horizontally on its own during peak (e.g., month-end close) without over-provisioning the Classification, Idle Detection, or Alerting services, which have very different load profiles.
- 2. Fault isolation across cloud providers and functions :** if one cloud provider changes its billing export format or its API becomes temporarily unavailable, only the corresponding ingestion path is affected; cost classification, idle-asset detection, alerting, and the FinOps dashboard continue operating on already-ingested data. A monolithic or tightly-coupled client-server design would risk a single failure taking down the whole portal.

Security Advantage

Each microservice holds only the credentials it needs e.g., only the Billing Ingestion Service is granted read-only cloud billing API/CSV access, and only the Payment-adjacent Alerting service can dispatch notifications — so a compromise of one service does not expose the credentials or data of the others. Service-to-service calls go through an API gateway with scoped tokens and mutual TLS, containing the blast radius of any single breach, which matters given the financial and infrastructure sensitivity of billing data.

Performance Benefit

Because ingestion, classification, and idle-detection run as separate services, each can be scaled and optimized independently to hit NFR-001's under-2-minute dashboard refresh even while parsing multi-gigabyte billing files for example, running several Billing Ingestion instances in parallel during month-end close instead of scaling an entire monolith to handle a load spike caused by just one component.

Multi-Cloud Asset Cost Optimization Portal UML Component Diagram (Microservices Architecture)

